HR & Payroll Module — Forms List & Implementation Plan

Project: Easy School Management System (School.API + School.Web) **Reference studied:** `D:\01 ZETA Office\02 Projects\01 ZETA Dropware` → `ZETAPOS/Views/HRM/{HrSetup,HrTransaction,HrReports}` and `ZETAPOS/Models/{HR*,Att*}.cs` **Date:** 2026-08-25

---

1. What already exists in EasySchool

Piece	Status	Location
Designation	Done (CRUD)	`DBModels/HR/Designation.cs`, `DesignationController`, `Views/Designation/Index.cshtml`
HR Grade	Done (CRUD)	`DBModels/HR/HRGrade.cs`, `HRGradeController`, `Views/HRGrade/Index.cshtml`
Employee	Partial — master exists but flat salary fields, `Department` is free text, no shift / machine card / off-days	`DBModels/HR/Employee.cs`, `Views/Employee/{Index,Form}.cshtml`
Salary	Skeleton — one flat row per employee/month, hard-coded allowance/deduction columns, no posting engine	`DBModels/HR/Salary.cs`, `SalaryRepository.cs` (275 lines)
Teacher	Separate entity (`SMSTeacher`)	`DBModels/HR/Teacher.cs`
HR Payroll sidebar section	Commented out	`School.API/Controllers/SidebarController.cs` ~line 152

Everything else below is new.

What the Dropware reference gives us (the proven flow)

```
HrAttMain (raw punches)  --AttendancePost-->  AttClosingDay (processed daily)
                                                      |
                                              --SalaryPost-->  AttClosingMonth (monthly salary)
```

Dropware keeps all the heavy computation in stored procedures (`SPHRAttClosingMonth`). EasySchool already mixes EF Core with `Database.SqlQueryRaw<T>()` + SPs in `School.API/Scripts/`, so we follow the same pattern: **setups/CRUD in EF Core, posting engines in stored procedures.**

---

2. Forms to add

2.1 Setups (`HR Payroll > Setups`)

#	Form	New/Existing	Notes
S1	**Department**	New	Replaces the free-text `Employee.Department`. Code, Name, HOD, Campus, Active.
S2	Designation	Existing	Add `DepartmentId` + `IsTeachingStaff` flag.
S3	HR Grade	Existing	Keep as-is; used for rule scoping.
S4	**Shift / Duty Timing**	New (key)	TimeIn, TimeOut, GraceMinutes, WorkingHours, MinHalfDayMinutes, MinFullDayMinutes, RestFrom/To, OffDay1, OffDay2. *This is where "when does late start" lives.*
S5	**Employee Shift Assignment**	New	Employee x Shift x EffectiveFrom/To (bulk assign by department).
S6	**Holiday Calendar**	New (key)	Date, Name, Type (Gazetted / Vacation / Weekly Off / Half Day), Campus, year-wise grid + "copy last year".

#	Form	New/Existing	Notes
S7	**Leave Type**	New	Casual, Sick, Annual, Short Leave, Without Pay, Maternity... with `IsPaid`, `DeductionDays`, `MaxPerYear`.
S8	**Leave Entitlement / Opening Balance**	New	Per employee (or per grade template) per session year.
S9	**Allowance & Deduction Heads**	New (key)	House Rent, Medical, Conveyance, EOBI, PF, Income Tax, Late Deduction, Absent Deduction, Fine, Bonus... Type = Allowance/Deduction, IsTaxable, IsFixed/Formula, PostingOrder. Replaces the hard-coded columns in `Salary.cs`.
S10	**Payroll Rule (Deduction/Allowance Formula)**	New (critical)	**The generic engine — see section 3.**
S11	**Payroll Rule Slabs**	New	Child grid of S10 (e.g. late 1–15 min = Rs 50, 16–30 = Rs 150, or "4th late = half day").
S12	**Late Exemption**	New	Employee + Date (or date range) + Reason -> skip late deduction. Mirrors Dropware's `AttLateExempt`.
S13	**Payroll Settings (Globals)**	New	MonthDaysBasis (Calendar / Fixed-30 / WorkingDays), salary cut-off day, rounding, OT multiplier, payslip header/footer, currency.
S14	**Salary Month Lock**	New	Lock/unlock a posted month so nobody edits attendance behind payroll.
S15	**Bank Setup**	New (small)	For salary transfer advice. Optional in Phase 4.

2.2 Day-to-Day / Transactions (`HR Payroll > Day-to-Day`)

#	Form	Notes
T1	**Employee Info** (enhance existing)	Add: DepartmentId, ShiftId, **MachineCardNo** (Hikvision person/card no.), OffDay1/2, IsStopSalary, IsOvertime, IsLateExempt, ProbationEndDate, EmployeeType, IBAN/Bank, EOBI/PF flags, SalaryTransferType.
T2	**Employee Salary Structure**	Effective-dated: Basic + N allowance/deduction lines from S9. Replaces the flat `BasicSalary/Allowances/Deductions` on `Employee`. Keeps salary history for increments.
T3	**Daily Attendance Entry (Manual)** (key)	**Phase-1 primary input.** Grid: pick Department + Date -> all employees listed with TimeIn / TimeOut / Status dropdown (P/A/L/HD/SL/Off/Holiday) / Remarks. Bulk "mark all present" + copy previous day.
T4	**Attendance Correction / Change**	Single-employee, single-date fix with reason + audit trail (who changed, old -> new).
T5	**Bulk Absent Marking**	Mark a set of employees absent for a date range.
T6	**Attendance Posting** (critical)	Runs the engine for a date range: pairs in/out, applies shift + grace, computes **LateMinutes, EarlyMinutes, WorkedMinutes, OverTime, HalfDay/Absent status**, honours holidays, weekly offs, approved leaves and late exemptions -> writes `HREmpAttendanceDaily`. Re-runnable (idempotent).
T7	**Leave Application**	Employee, LeaveType, From/To, Days, Reason, attachment.
T8	**Leave Approval**	Approver queue: approve / reject / partial, updates balance and feeds T6.
T9	**Monthly Allowance / Deduction Entry**	One-off amounts for a month: bonus, fine, arrears, mobile bill.
T10	**Advance Entry**	Salary advance, deducted in full next month.
T11	**Loan Entry + Approval**	Amount, installments, start month -> auto-generates installment schedule.
T12	**Salary Posting (Generate Payroll)** (critical)	Pick Month + Department/Employee -> engine builds `HRSalaryMonth` header + detail lines from: salary structure, attendance summary, payroll rules (late/absent deduction), leaves, loans, advances, one-off entries, tax. Re-postable until month is locked.
T13	**Salary Sheet Review / Adjustment**	Editable grid of the posted month before finalising; shows every allowance/deduction column, per-employee override with reason.

#	Form	Notes
T14	**Salary Payment / Disbursement**	Mark paid: Cash / Bank / Cheque, payment date, reference. Partial payment supported.
T15	**Employee Increment / Salary Revision**	Creates a new effective-dated row in T2.
T16	**Final Settlement** (later)	Resignation: unpaid salary + leave encashment - loan balance.

2.3 Reports (`HR Payroll > Reports`)

#	Report	Notes
R1	Daily Attendance Report	Department/date-wise, present/absent/late counts.
R2	Monthly Attendance Summary	Employee x day matrix (P/A/L/H/O) + totals. Classic HR sheet.
R3	**Late Comers Report**	Date range, late minutes, computed deduction — the client will use this daily.
R4	Absentee Report	
R5	Leave Ledger / Balance Report	Opening, availed, balance per leave type.
R6	**Salary Sheet**	Month + department, all heads as columns, grand totals. Excel export.
R7	**Salary Slip / Payslip**	Print (PDF via existing `PuppeteerPdfRenderer`) **and send via WhatsApp** using the existing WAHA sender — infrastructure already there.
R8	Bank Transfer Advice	Bank-wise list with IBAN + net amount.
R9	Loan / Advance Ledger	Outstanding balances.
R10	Employee Master / Directory Report	
R11	EOBI / PF Statement	
R12	Employee Card Print	(Dropware has this; low priority.)

Total: ~15 setup forms + ~16 transaction forms + ~12 reports.

---

3. The generic deduction engine (the key design decision)

Sir Syed Model School deducts **per minute of late arrival**. Other schools will not. So the formula must be **data, not code**.

3.1 `HRPayrollRule` table

Column	Purpose
`Id`, `Code`, `Name`, `IsActive`	
`RuleType`	`LateComing`, `EarlyGoing`, `Absent`, `HalfDay`, `ShortHours`, `Overtime`, `Allowance`
`AllowDedHeadId`	Which head (S9) the computed amount posts to
`CalcMethod`	`PerMinute`, `PerHour`, `PerDay`, `FixedAmount`, `PercentOfBasic`, `Slab`, `Expression`
`RateSource`	`Fixed` (rate typed in) **or** `Derived` (computed from salary)
`FixedRate`	Rs per minute/hour/day/occurrence when `RateSource = Fixed`
`BasisAmount`	`Basic`, `Gross`, `Net` — the salary base when `RateSource = Derived`

Column	Purpose
`MonthDaysBasis`	`Calendar` , `Fixed30` , `WorkingDays`
`HoursPerDaySource`	`FromShift` or a fixed number
`GraceMinutes`	Free minutes before the rule bites (can also live on the shift; rule wins)
`FreeOccurrencesPerMonth`	e.g. first 3 lates forgiven
`MinAmount` / `MaxAmountPerDay` / `MaxAmountPerMonth`	Caps
`Expression`	Optional formula string for exotic cases (see 3.4)
`ScopeType` + `ScopeId`	`Global` / `Campus` / `Department` / `Designation` / `Grade` / `EmployeeType` / `Employee`
`Priority`	Tie-breaker
`EffectiveFrom` / `EffectiveTo`	Rules change; history must stay reproducible

`HRPayrollRuleSlab` (child): `FromValue` , `ToValue` , `Amount` **or** `Days` **or** `Percent` . Used for "1–15 min -> Rs 100, 16–30 min -> Rs 250, >30 min -> half day".

3.2 Rule resolution

Most-specific-wins, evaluated at posting time against the salary month's date:

```
Employee > Grade > Designation > Department > Campus > Global
```

...filtered by `EffectiveFrom <= MonthEnd AND (EffectiveTo IS NULL OR EffectiveTo >= MonthStart)` , then ordered by `Priority` .

3.3 Worked example — Sir Syed Model School

Setup one rule:

```
RuleType          = LateComing
CalcMethod        = PerMinute
RateSource        = Derived
BasisAmount       = Basic
MonthDaysBasis    = Fixed30
HoursPerDaySource = FromShift       (say 7 hrs)
GraceMinutes      = 10
MaxAmountPerDay   = 500
ScopeType         = Global
```

Engine computes:

```
PerMinuteRate  = Basic / 30 / 7 / 60
ChargeMinutes  = MAX(0, LateMinutes - GraceMinutes)
Deduction      = MIN(ChargeMinutes * PerMinuteRate, MaxAmountPerDay)
```

Another school charging a **flat Rs 20 per late minute** only changes `RateSource = Fixed, FixedRate = 20` . A school that deducts **half a day after 3 lates** switches to `CalcMethod = Slab` with `FreeOccurrencesPerMonth = 3` and a slab row of `Days = 0.5` . **No code change in any of these cases.**

3.4 Escape hatch

`CalcMethod = Expression` evaluates `Expression` with **NCalc** (`dotnet add School.API package NCalc`) against variables: `LateMinutes` , `EarlyMinutes` , `AbsentDays` , `LateCount` , `Basic` , `Gross` , `MonthDays` , `WorkingDays` , `HoursPerDay` , `PerDayRate` , `PerMinuteRate` .

Example: `if(LateCount > 3, PerDayRate * 0.5, LateMinutes * PerMinuteRate)`

Keep this to Phase 3+; the structured methods cover the large majority of real schools.

---

4. Attendance input strategy (Hikvision)

Decision: **manual entry now, Hikvision import later.** The schema is designed so the import drops in without a migration.

> **Important:** the same Hikvision machine will later feed **student attendance as well as staff attendance**. The punch-import layer must therefore be built generic from day one — see 4.2.

4.1 Staff attendance tables

Two tables from day one:

1. `HRAttendanceRaw` — one row per punch: `PersonType` (Staff/Student), `PersonId`, `MachineCardNo`, `PunchDateTime`, `Direction` (In/Out/Unknown), `DeviceCode`, `Source` (`Manual` / `Import` / `Device`), `BatchId`. Phase 1 writes to it from the manual form; Phase 6 writes to it from the file importer. Named without `Emp` on purpose — student punches land here too (see 4.2).
2. `HREmpAttendanceDaily` — one row per employee per day, produced by **Attendance Posting** (T6): `AttDate`, `ShiftId`, `TimeIn`, `TimeOut`, `WorkedMinutes`, `LateMinutes`, `EarlyMinutes`, `OverTimeMinutes`, `Status`, `LeaveTypeId`, `IsPayable`, `IsOff`, `IsHoliday`, `IgnoreLate`, `Remarks`, audit columns. This is what payroll reads. (Mirrors Dropware's `AttClosingDay`.)

Phase 6 importer: upload the iVMS-4200 / Hikvision export (CSV or XLSX — add `ClosedXML`), map `Person ID` / `Card No` -> `Employee.MachineCardNo`, preview grid with unmatched rows flagged, then commit to `HRAttendanceRaw` under one `BatchId` (re-importable / reversible). Posting (T6) is unchanged.

4.2 Student attendance from the same machine (build the importer generic)

Later the school will also pull **student** attendance off the Hikvision device. The device does not care whether a card belongs to a teacher or a student — it emits the same punch record. So the importer must not be written as an HR-only feature.

Design rule: one raw punch table, one importer, two posting engines.

```
Hikvision export (CSV/XLSX)
        |
        v
  AttendanceImportService          <-- shared, PersonType-aware
        |
        v
  HRAttendanceRaw  (PersonType = Staff | Student, PersonId, MachineCardNo, PunchDateTime, ...)
        |                                  |
   Staff punches                      Student punches
        |                                  |
        v                                  v
  SpPost_HRAttendanceDaily          SpPost_StdAttendanceDaily
        |                                  |
        v                                  v
  HREmpAttendanceDaily                StdAttendence  (existing table)
```

Concrete changes to make **now**, in Phase 0/1, so Phase 6 is a drop-in:

Item	Do it now
Name the raw table generically	Call it `HRAttendanceRaw` — **not** `HREmpAttendanceRaw`
Add `PersonType`	`1 = Staff`, `2 = Student` — first column of the natural key
Add `PersonId`	Resolves to `Employees.Id` or `Students.Id` depending on `PersonType`
Add `MachineCardNo` to `Student`	Same column already planned for `Employee` (T1). Students currently key on `AdmissionNo` / `RollNo` / `ClassRollNo` — none of these is the device card number
Namespace the service	`Services/Implementations/Attendance/AttendanceImportService.cs`, not under `HR/`

Item	Do it now
Keep `StdAttendence` as the student target	The table already exists (`DBModels/Academic/StdAttendence.cs`) with `IsLate` , `ISentSMS` , `Remarks` — the student posting engine writes into it, so the existing Mark Attendance screens, reports and the SMS/WhatsApp absent notification keep working unchanged

Student-side benefits that fall out for free once the device feed exists:

- **Auto absent/late WhatsApp to parents** — the existing `AbsentNotificationJob` + WAHA pipeline already reads `StdAttendence` ; it starts firing off real gate data instead of manual marking.
- Late-arrival tracking for students (`StdAttendence.IsLate` is already there and unused by the manual form).
- No student *deduction* logic is needed — the payroll rule engine (section 3) stays staff-only.

Effort delta if this is designed in from the start: **negligible** (~2 extra columns + a namespace choice). Effort if retrofitted later: a data migration on the raw table plus a rewrite of the importer.

---

5. New tables (summary)

```
Setups        HRDepartment, HRShift, HREmployeeShift, HRHoliday, HRLeaveType,
              HRLeaveEntitlement, HRAllowDedHead, HRPayrollRule, HRPayrollRuleSlab,
              HRLateExemption, HRPayrollSetting, HRSalaryMonthLock, HRBank

Attendance    HRAttendanceRaw, HREmpAttendanceDaily

Leave         HRLeaveApplication

Payroll       HREmployeeSalaryStructure, HREmployeeSalaryStructureDetail,
              HRMonthlyAllowDed, HRAdvance, HRLoan, HRLoanInstallment,
              HRSalaryMonth, HRSalaryMonthDetail, HRSalaryPayment
```

Stored procedures (in `School.API/Scripts/` , following the existing `SpXxx_` naming):

- `SpPost_HRAttendanceDaily` — the T6 posting engine
- `SpPost_HRSalaryMonth` — the T12 payroll engine
- `SpGet_HRPayrollRuleResolve` — rule resolution helper
- `SpRPT_HRAttendanceMonthly` , `SpRPT_HRLateComers` , `SpRPT_HRSalarySheet` , `SpRPT_HRPayslip`

---

6. Phased implementation plan

Each feature follows the 8-step recipe in `CLAUDE.md` (model -> DbSet -> DTO -> repo -> service -> `Program.cs` `AddScoped` -> API controller -> Web MVC controller + view).

Phase 0 — Foundation (1–2 days)

- [ ] Uncomment + expand the **HR Payroll** section in `SidebarController.cs` (Setups / Day-to-Day / Reports sub-menus)
- [ ] Register HR pages in `Menus` + `RolePermissions` so RBAC works
- [ ] `HRDepartment` setup (S1) — full stack, use `PaymentMethod` as the template (newest clean CRUD in the repo)
- [ ] Enhance `Employee` (T1): DepartmentId, ShiftId, MachineCardNo, OffDay1/2, IsStopSalary, IsLateExempt, ProbationEndDate, bank fields
- [ ] `Scripts/HRPayroll_CreateTables.sql` for the whole module (single idempotent script)

Deliverable: menu visible, departments manageable, employee master payroll-ready.

Phase 1 — Attendance core (4–6 days) — highest value

- [ ] `HRShift` setup (S4) + `HREmployeeShift` assignment (S5)

- [ ] `HRHoliday` calendar (S6) with year grid + copy-previous-year
- [ ] `HRAttendanceRaw` (with `PersonType` / `PersonId` — see 4.2) + `HREmpAttendanceDaily` tables
- [ ] Add `MachineCardNo` to `Student` too, so the device feed can resolve students later
- [ ] **Daily Attendance Entry** manual grid (T3) — the form the client uses every morning
- [ ] Attendance Correction (T4) + Bulk Absent (T5)
- [ ] `SpPost_HRAttendanceDaily` + **Attendance Posting** screen (T6) — late/early/short-hours computation
- [ ] R1 Daily Attendance, R2 Monthly Summary, R3 Late Comers

Deliverable: the school can run attendance end-to-end without payroll.

Phase 2 — Leave management (3–4 days)

- [ ] Leave Type (S7) + Entitlement (S8)
- [ ] Leave Application (T7) + Approval (T8)
- [ ] Feed approved leaves into `SpPost_HRAttendanceDaily`
- [ ] R5 Leave Ledger

Phase 3 — Payroll setups & the rule engine (4–5 days) — the generic part

- [ ] `HRAllowDedHead` (S9)
- [ ] `HRPayrollRule` + `HRPayrollRuleSlab` (S10, S11) — form with a **live preview calculator** ("employee X, 25 min late -> Rs Y") so the client can verify the formula before it touches payroll
- [ ] Late Exemption (S12)
- [ ] Payroll Settings (S13)
- [ ] `HREmployeeSalaryStructure` (T2) — migrate existing flat `Employee` salary fields into it
- [ ] `SpGet_HRPayrollRuleResolve` + unit-testable C# `PayrollRuleEvaluator` service

Deliverable: Sir Syed's per-minute rule configured through the UI; another school's rule is a data change.

Phase 4 — Payroll processing (5–7 days)

- [ ] `HRSalaryMonth` + `HRSalaryMonthDetail`
- [ ] `SpPost_HRSalaryMonth` + **Salary Posting** screen (T12)
- [ ] Salary Sheet Review / Adjustment (T13)
- [ ] Salary Payment (T14) + Month Lock (S14)
- [ ] R6 Salary Sheet, R7 Payslip (PDF via `PuppeteerPdfRenderer`), R8 Bank Advice
- [ ] Retire the old flat `Salary` entity (keep the table, stop writing to it)

Phase 5 — Loans, advances & increments (2–3 days)

- [ ] Advance (T10), Loan + Approval + installments (T11)
- [ ] Monthly one-off Allowance/Deduction (T9)
- [ ] Increment / Salary Revision (T15)
- [ ] R9 Loan Ledger

Phase 6 — Hikvision import (staff) & extras (3–4 days)

- [ ] Shared `AttendanceImportService`: CSV/XLSX importer for `HRAttendanceRaw` (add `ClosedXML`), mapping preview, unmatched-card report, batch reversal. **Written PersonType-aware from the start**
- [ ] Staff card mapping (`Employee.MachineCardNo`) + wire into `SpPost_HRAttendanceDaily`
- [ ] **Payslip over WhatsApp** through the existing `ISmsSender` / WAHA pipeline — reuse `SmsTemplateCatalog`

- [ ] R10 Employee Directory, R11 EOBI/PF, R12 Card Print
- [ ] Optional: absent-notification job for staff (mirror the existing `AbsentNotificationJob`)

Phase 7 — Student attendance from the machine (2–3 days)

Only reachable cheaply because 4.2 was designed in during Phase 1.

- [ ] Student card enrolment screen: bulk-assign `MachineCardNo` to students (class/section-wise grid)
- [ ] `SpPost_StdAttendanceDaily` — pairs student punches, applies school timing + grace, writes `StdAttendence` (sets `Attendence`, `IsLate`, `Remarks`)
- [ ] Student Attendance Posting screen (date range, per class/section, re-runnable)
- [ ] Reconciliation view: device punches vs manually marked attendance, with an override winner rule
- [ ] Point the existing `AbsentNotificationJob` at posted device data so parent WhatsApp alerts go out automatically instead of depending on manual marking
- [ ] Student late-comers report (`IsLate` is already on `StdAttendence` and currently unused)

Rough total: 24–34 working days.

---

7. Decisions to confirm before Phase 1

1. **Teacher vs Employee** — `SMSTeacher` and `Employees` are separate tables today. Do teachers get payroll? If yes, either merge teachers into `Employees` (recommended) or add `Employee.TeacherId` link.
2. **Month days basis for Sir Syed** — fixed 30, or calendar days (28/30/31)?
3. **Working hours per day** — from the shift, or a school-wide constant?
4. **Grace period** — how many minutes before late counts? Is there a daily cap on late deduction?
5. **Weekly off** — Sunday only, or Friday half-day + Sunday?
6. **Summer/winter timings** — does the shift change seasonally? (Dropware handles this with "DayLight" columns on the shift — worth copying if yes.)
7. **Campus scoping** — should HR data be per-campus like `FeeStructure`, or school-wide?
8. **Student device attendance (4.2)** — one machine for staff and students, or separate devices per gate? And when a student is punched by the device *and* marked manually by the class teacher, which wins? (Recommendation: device wins for time-in/late, manual wins for absent, with a reconciliation screen.)
